

Unidad 4 – parte 2

Agregaciones

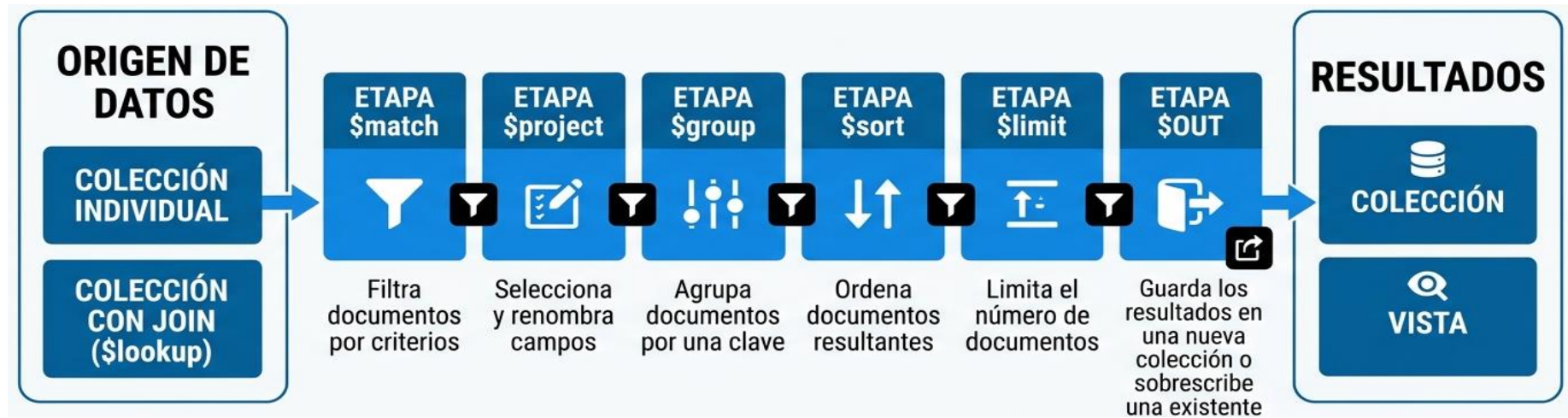
Bases de Datos Aplicada
v3.0 –2026

❑ Contenidos de esta clase:

- ❖ Aggregation Framework: concepto y estructura de pipeline
- ❖ Etapas de transformación
- ❖ Etapas de ordenamiento y control
- ❖ Guardar, exportar y crear vistas desde una agregación
- ❖ Etapas de filtrado y agrupamiento
- ❖ Acumuladores

Definiciones

- ❑ Un **Framework** se puede ver como una caja de herramientas organizada que nos da métodos, reglas y estructuras ya listas para resolver un tipo de problema.
- ❑ **Aggregation Framework** utiliza una estructura de canalizaciones o flujo de etapas, denominadas (pipeline).
- ❑ En cada etapa se pueden filtrar, ordenar, agrupar, remodelar y modificar los documentos que pasan por el proceso.



Nota: \$out sobrescribe la colección de destino si ya existe.

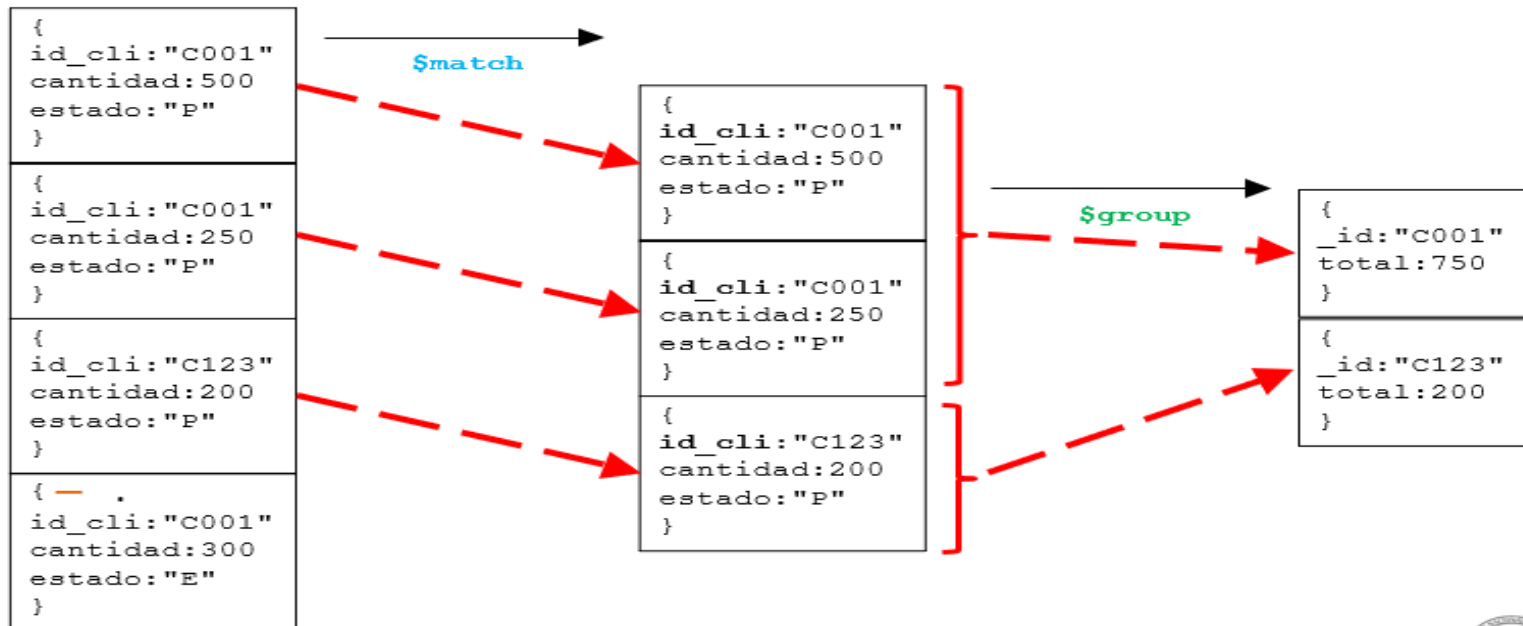
Canalizaciones

- ❑ Ejemplo: Filtrar únicamente las ventas cuyo estado sea "P" (pendiente). Calcular el total de cantidad de productos comprados por cada cliente.

db.collection.aggregate([<etapa1>,...])

db.ventas.aggregate([
 {**\$match**:{estado:"P"}},
 {**\$group**:{_id:"\$id_cli", **total**:{\$sum:"\$cantidad"}}}])

Acumulador



❑ Reglas importantes

- Las etapas pueden repetirse (ejemplo: varios filtros en distintos momentos).
- La entrada suele ser una colección (un conjunto de documentos).
- Una consulta compleja se puede dividir en varias etapas más simples.
- La salida de cada etapa será la entrada de la siguiente.

¿Que permiten una hacer las agregaciones?

❑ Por ejemplo, de estas dos Colecciones, obtener el siguiente resultado:

Colección datosventas

```
_id: ObjectId('68b4235d87547514c3b456cd')
id_venta : 1
fecha : "2024-05-16"
cantidad : 1
importe : 1500
▼ cliente : Object
  id : 1
  nombre : "Juan"
  apellido : "García López"
  direccion : "Calle 123"
  localidad : "Ramos Mejía"
id_vendedor : 1
▼ producto : Object
  id : 101
  nombre : "Laptop"
  marca : "Dell"
```

Colección datosvendedores

```
_id: ObjectId('68b4232e87547514c3b456c0')
Id : 1
nombre : "Juan"
apellidos : "García López"
edad : 30
sexo : "masculino"
ciudad : "Ramos Mejía"
pais : "Argentina"
```

Resultado del proceso de reunión.

Output after \$project stage (Sample of 10 documents)

```
id_venta : 1
fecha : "2024-05-16"
cantidad : 1
importe : 1500
cliente_id : 1
cliente_nombre : "Juan García López"
cliente_direccion : "Calle 123"
cliente_localidad : "Ramos Mejía"
producto_id : 101
producto_nombre : "Laptop"
producto_marca : "Dell"
vendedor_id : 1
vendedor_nombre : "Juan"
vendedor_apellidos : "García López"
vendedor_edad : 30
vendedorsexo : "masculino"
vendedor_ciudad : "Ramos Mejía"
vendedor_pais : "Argentina"
Total_Importe : 1500
mes : "Mayo"
fecha_formateada : "16/05/2024"
fechaConvertida : 2024-05-16T00:00:00.000+0...
fechaActualFormateada : "2025-08-
31T16:51:59.699Z"
diasPrestamo : 472
```

Agregaciones

- ❑ Esta es la pantalla principal. Permite entre otras:
 - Crear y modificar etapas.
 - Visualizar los datos que resulten de cada etapa en tiempo real, lo que facilita el ajuste y la depuración de consultas.

The screenshot shows the MongoDB Aggregations interface for a collection named **ejemplo.Datosvendedores**. The interface includes tabs for **Documents**, **Aggregations**, **Schema**, **Indexes**, and **Validation**. The **Aggregations** tab is active, showing a pipeline that is currently empty. There are buttons for **Explain**, **Export**, **Run**, and **More Options**. Below the pipeline, there are buttons for **SAVE**, **CREATE NEW**, and **EXPORT TO LANGUAGE**. The preview section shows three sample documents with fields like **_id**, **Id**, **nombre**, **apellidos**, **edad**, **sexo**, **ciudad**, and **pais**.

ejemplo.Datosvendedores 12 DOCUMENTS 1 INDEXES

Documents **Aggregations** Schema Indexes Validation

Pipeline Your pipeline is currently empty. Need help getting started? [Generate aggregation](#) **Explain** **Export** **Run** [More Options](#)

Untitled **SAVE** **+ CREATE NEW** **EXPORT TO LANGUAGE** **PREVIEW** **STAGES** **TEXT**

12 Documents in the collection

Preview of documents

```
_id: ObjectId('6640d7f340d1a3a121cdb584')
Id: 1
nombre: "Juan"
apellidos: "García López"
edad: 30
sexo: "masculino"
ciudad: "Ramos Mejía"
pais: "Argentina"
```

```
_id: ObjectId('6640d7f340d1a3a121cdb585')
Id: 2
nombre: "María"
apellidos: "Martínez Fernández"
edad: 25
sexo: "femenino"
ciudad: "San Justo"
pais: "Argentina"
```

```
_id: ObjectId('6640d7f340d1a3a121cdb586')
Id: 3
nombre: "Carlos"
apellidos: "Gómez"
edad: 35
sexo: "masculino"
ciudad: "Isidro Casanova"
pais: "Argentina"
```

+ Add Stage

[Learn more about aggregation pipeline stages](#)

Crear agregaciones

- ❑ Se maneja a través de esta interfaz gráfica que facilita la creación de las diferentes etapas (stage) de agregación.

+ Add Stage

Este botón, se utiliza para comenzar o para insertar una nueva etapa.

The screenshot displays the MongoDB Aggregations interface. At the top, there are tabs for Documents (0), Aggregations (selected), Schema, Indexes (1), and Validation. Below the tabs, a message states 'Your pipeline is currently empty. [Generate aggregation](#) +'. To the right of this message are buttons for 'Explain', 'Export', 'Run', and 'Options'. Below the message bar, there are buttons for 'SAVE', 'CREATE NEW', and 'EXPORT TO LANGUAGE'. To the right of these buttons are a 'PREVIEW' toggle, a 'STAGES' button, a 'TEXT' button, and a 'WIZARD' button. The main area shows a 'Preview of documents' section. Below this, there is a 'Stage 1' dropdown menu with a 'Select' button. A dropdown menu is open, showing a list of aggregation stages with their descriptions:

Stage	Description
\$addFields	Adds new field(s) to a document with a computed value, or reassigns an existing field(s) with a computed value.
\$bucket	Categorizes incoming documents into groups, called buckets, based on specified boundaries.
\$bucketAuto	Automatically categorizes documents into a specified number of buckets, attempting even distribution if possible.
\$collStats	Returns statistics regarding a collection or view.
\$count	Returns a count of the number of documents at this stage of the aggregation pipeline.

At the bottom of the interface, there is a '+ Add Stage' button and a link to 'Learn more about aggregation pipeline stages'.

Algunas de las etapas del pipeline de agregación

❑ Filtrado (Filtering)

Sirven para seleccionar documentos dentro del pipeline. seleccionar documentos.

Ejemplo: `{ $match: { estado: "activo" } }`

Etapas	Descripción
<code>\$match</code>	Filtra documentos (equivalente a WHERE)
<code>\$redact</code>	Filtra datos a nivel de documento/campo según lógica

❑ Transformación / Proyección

Modifican la estructura de los documentos

Ejemplo: `{ $project: { nombre: 1, total: { $multiply: ["$precio", "$cantidad"] } } }`

Etapas	Descripción
<code>\$project</code>	Incluye, excluye o calcula campos
<code>\$set / \$addFields</code>	Agrega nuevos campos
<code>\$unset</code>	Elimina campos
<code>\$replaceRoot</code>	Reemplaza el documento completo
<code>\$replaceWith</code>	Igual que replaceRoot pero más moderno

Algunas de las principales etapas son:

☐ Unión / Join

Relacionan colecciones

Etapa	Descripción
<code>\$lookup</code>	JOIN con otra colección
<code>\$graphLookup</code>	JOIN recursivo (relaciones jerárquicas)

☐ Agrupación

Modifican la estructura de los documentos

Etapa	Descripción
<code>\$group</code>	Agrupar documentos y aplica acumuladores

☐ Descomposición / Arrays

Trabajan con arrays

Etapa	Descripción
<code>\$unwind</code>	Descompone arrays en múltiples documentos

Algunas de las principales etapas son:

❑ Ordenamiento y límite

Controlan el orden y cantidad.

Etapa	Descripción
\$sort	Ordena documentos
\$limit	Limita cantidad
\$skip	Salta documentos

❑ Cálculo / Métricas avanzadas

Etapas analíticas

Etapa	Descripción
\$count	Cuenta documentos
\$sortByCount	Combina agrupación + orden
\$bucket	Clasifica en rangos
\$bucketAuto	Rangos automáticos
\$facet	Múltiples pipelines paralelos
\$setWindowFields	Funciones ventana (tipo SQL OVER)



AGREGACIÓN DE MONGODB:

HACER UN JOIN ENTRE DOS COLECCIONES CON \$lookup



1) Colección 'PEDIDOS' (orders - azul/verde)

Pedido #1

```
{
  _id: "1d70874525970838655333",
  cliente_id: 101,
  fecha: "2023-01-21 12:00:03",
  total: $150.00
},
```

Pedido #2

```
{
  _id: "1d70874525970838655333",
  cliente_id: 102,
  fecha: "2023-01-21 12:00:02",
  total: $150.00
}
```

2) Colección 'CLIENTES' (customers - amarillo/naranja)

Cliente #101

```
{
  _id: "1d70874525970838653101",
  nombre: "Ana García",
  email: "ana@ejemplo.com",
  ciudad: "CDMX"
},
```

Cliente #102

```
{
  _id: "1d79874525970839653102",
  nombre: "Luis Pérez",
  email: "elisat@ejemplo.com",
  ciudad: "CDMX"
}
```



```
{
  _id: "1d70874525970838653367",
  cliente_id: 101,
  total: $150.00,
  cliente_info: {
    _id: 101,
    nombre: "Ana García",
    email: "ana@ejemplo.com",
    ciudad: "CDMX"
  }
}
```

Agregación Resulto

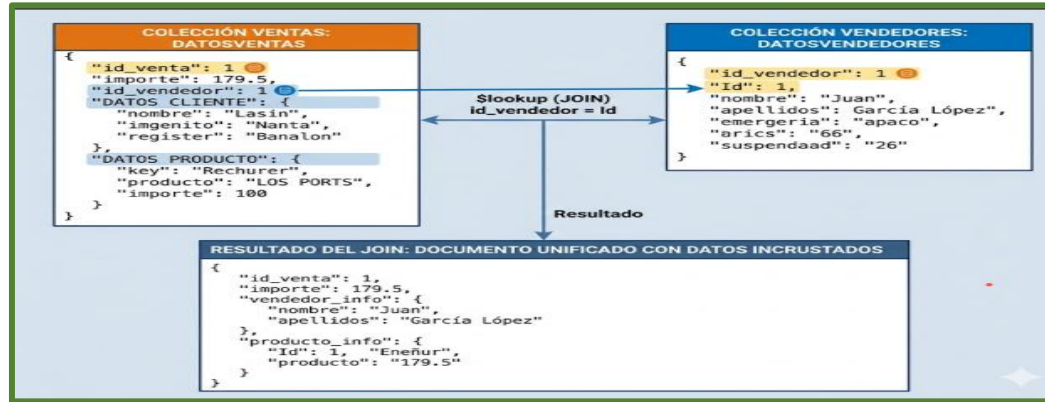
'DOCUMENTO FINAL FUSIONADO'
(Colección 'PEDIDOS' con Datos de Cliente)

```
{
  _id: "1d70874525970838653367",
  cliente_id: 101,
  total: $150.00,
  cliente_info: {
    _id: 101,
    nombre: "Ana García",
    email: "ana@ejemplo.com",
    ciudad: "CDMX"
  }
}
```

```
13 orders {
14   localField: "Ana Pérez",
15   nombres: "ana@ejemplo.com",
16   $lookup:
17     from: "clientes",
18     foreignField: "_id",
19     as: "cliente_info"
20 }
```

\$lookup

- ❑ Permite hacer una combinación entre dos colecciones (join).
- ❑ Añade un array a cada documento que contiene los elementos coincidentes de la colección enlazada..



Stage1 \$lookup

```
1 /**
2  * from: The target collection.
3  * localField: The local join field.
4  * foreignField: The target join field.
5  * as: The name for the results.
6  * pipeline: Optional pipeline to run on the
7  * let: Optional variables to use in the pipeline.
8  */
9 {
10   from: "datosvendedores",
11   localField: "id_vendedor",
12   foreignField: "Id",
13   as: "vendedor"
14 }
```

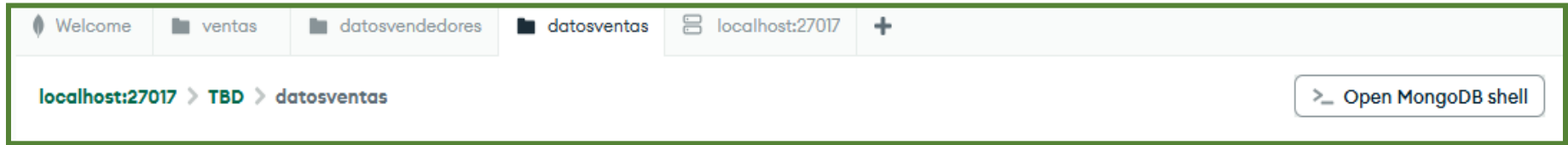
Output after \$lookup stage (Sample of 10 documents)

```
_id: ObjectId('66d19b677b6712922102217f')
id_venta: 1
fecha: "2024-05-16"
cantidad: 1
importe: 1500
cliente: Object
  id_vendedor: 1
producto: Object
  Id: 1, "Enefur"
vendedor: Array (1)

_id: ObjectId('66d19b677b6712922102217f')
id_venta: 2
fecha: "2024-05-15"
cantidad: 2
importe: 3000
cliente: Object
  id_vendedor: 2
producto: Object
  Id: 1, "Enefur"
vendedor: Array (1)
```

MongoDB Shell

- ❑ Dentro de Compass puedo ejecutar el scripts:



```
db.datosventas.aggregate([
{
  $lookup: {
    from: "datosvendedores",
    // Colección con la que haces join
    localField: "id_vendedor",
    // Campo en la colección "ventas"
    foreignField: "Id",
    // Campo en la colección "datosvendedores"
    as: "vendedor"
  }
}]
```

```
>_MONGOSH
> db.datosventas.aggregate([{$lookup: {from: "datosvendedores", localField: "id_vendedor",
foreignField: "Id", as: "vendedor" }}}])
< {
  _id: ObjectId('68b4235d87547514c3b456cd'),
  id_venta: 1,
  fecha: '2024-05-16',
  cantidad: 1,
  importe: 1500,
  cliente: {
    id: 1,
    nombre: 'Juan',
    apellido: 'García López',
    direccion: 'Calle 123',
    localidad: 'Ramos Mejía'
  },
  id_vendedor: 1,
  producto: {
    id: 101,
    nombre: 'Laptop',
    marca: 'Dell'
  },
  vendedor: [
    {
      _id: ObjectId('68b4232e87547514c3b456c0'),
      Id: 1,
      nombre: 'Juan',
      apellidos: 'García López',
      edad: 30,
      sexo: 'masculino',
      ciudad: 'Ramos Mejía',
```

\$unwind (Descomposición de Arrays)

- ❑ Este comando descompone un documento en tantos documentos como elementos tenga el array.
- ❑ Cada elemento del array genera un nuevo documento, duplicando el resto de los campos del documento original
- ❑ **Sintaxis extendida.**

```
$unwind: {  
    path: "$campoArray",  
    preserveNullAndEmptyArrays: true,  
    includeArrayIndex: "indice" }
```

❑ Donde:

- **path:** Campo array que se va a descomponer.
- **preserveNullAndEmptyArrays:** (Opcional)
 - true → mantiene documentos sin array o con array vacío
 - false → los descarta (valor por defecto)
- **includeArrayIndex:** (Opcional)

Agrega un campo con la **posición** del elemento dentro del array. Si no es un array, tendrá un valor **null**.

❑ Ejemplo:

```
{"_id": 1, "item": "ABC1", "tallas": ["S","M","L"]}
```

```
db.inventario.aggregate([{$unwind : "$tallas" }])
```

```
{ "_id": 1, "item": "ABC1", "tallas": "S" }  
{ "_id": 1, "item": "ABC1", "tallas": "M" }  
{ "_id": 1, "item": "ABC1", "tallas": "L" }
```

\$unwind (Descomposición de Arrays)

- ❑ Con todos los parámetros.

Ejemplo: {"_id": 1, "item": "ABC1", "tallas": ["S","M","L"]}

```
db.inventario.aggregate ([
  { $unwind:
    { path: "$tallas",
      includeArrayIndex: "indice",
      preserveNullAndEmptyArrays: true
    }
  }
])
```

```
{ "_id": 1, "item": "ABC1", "tallas": "S", "indice": 0 }
{ "_id": 1, "item": "ABC1", "tallas": "M", "indice": 1 }
{ "_id": 1, "item": "ABC1", "tallas": "L", "indice": 2 }
```

- ❑ **\$unwind** transforma los elementos de un array en documentos individuales dentro del pipeline de agregación.
- ❑ El campo "**indice**" indica la posición original del elemento dentro del array (comienza en 0).
- ❑ **\$unwind** se utiliza antes de \$group para poder agrupar elementos que originalmente estaban dentro de arrays.

\$unwind

- ❑ En este ejemplo, separa los documentos de la colección original que contienen un array en documentos individuales para cada vendedor.

The screenshot shows the MongoDB Compass interface. On the left, the 'Pipeline' tab is active, displaying a \$unwind stage configuration. The configuration is as follows:

```
4 * foreignField: The target join field.
5 * as: The name for the results.
6 * pipeline: Optional pipeline to run on the output of the unwind.
7 * let: Optional variables to use in the pipeline.
8 */
9 {
10   from: "datosvendedores",
11   localField: "id_vendedor",
12   foreignField: "Id",
13   as: "vendedores"
14 }
```

On the right, the 'Output' tab shows the result of the \$unwind stage. It displays a single document with the following structure:

```
{
  "producto": {
    "nombre": "Producto A",
    "precio": 100
  },
  "vendedores": [
    {
      "Id": 1,
      "nombre": "Juan",
      "apellidos": "García López",
      "edad": 30,
      "sexo": "masculino",
      "ciudad": "Ramos Mejía"
    }
  ]
}
```

- Cuando se utiliza el operador \$unwind, el parámetro path debe hacer referencia al nombre del campo a desanidar, y para ello siempre se antepone el símbolo \$ para indicar que es un campo del documento, y no una cadena de texto literal.

The screenshot shows the MongoDB Compass interface. At the top, 'Stage 2' is selected, and the stage is set to '\$unwind'. The configuration is as follows:

```
1 /**
2  * path: Path to the array field.
3  * includeArrayIndex: Optional name for the array index field.
4  * preserveNullAndEmptyArrays: Optional
5  *   toggle to unwind null and empty values as documents.
6  */
7 {
8   path: "$vendedores",
9   preserveNullAndEmptyArrays: true
10 }
```

On the right, the 'Output' tab shows the result of the \$unwind stage. It displays a single document with the following structure:

```
{
  "vendedores": {
    "_id": ObjectId('68272fc21a30ba135ae7ad57'),
    "Id": 1,
    "nombre": "Juan",
    "apellidos": "García López",
    "edad": 30,
    "sexo": "masculino",
    "ciudad": "Ramos Mejía",
    "pais": "Argentina"
  }
}
```


¿Qué hace `preserveNullAndEmptyArrays`?

❑ Ejemplo anterior

```
db.datosventas.aggregate( [  { $lookup: {from: "datosvendedores",  
                                         localField: "id_vendedor",  
                                         foreignField: "Id", as: "vendedor" } },  
  { $unwind: { path: "$vendedor", preserveNullAndEmptyArrays: true} } ] )
```

❑ Cuando se utiliza `$unwind`, MongoDB:

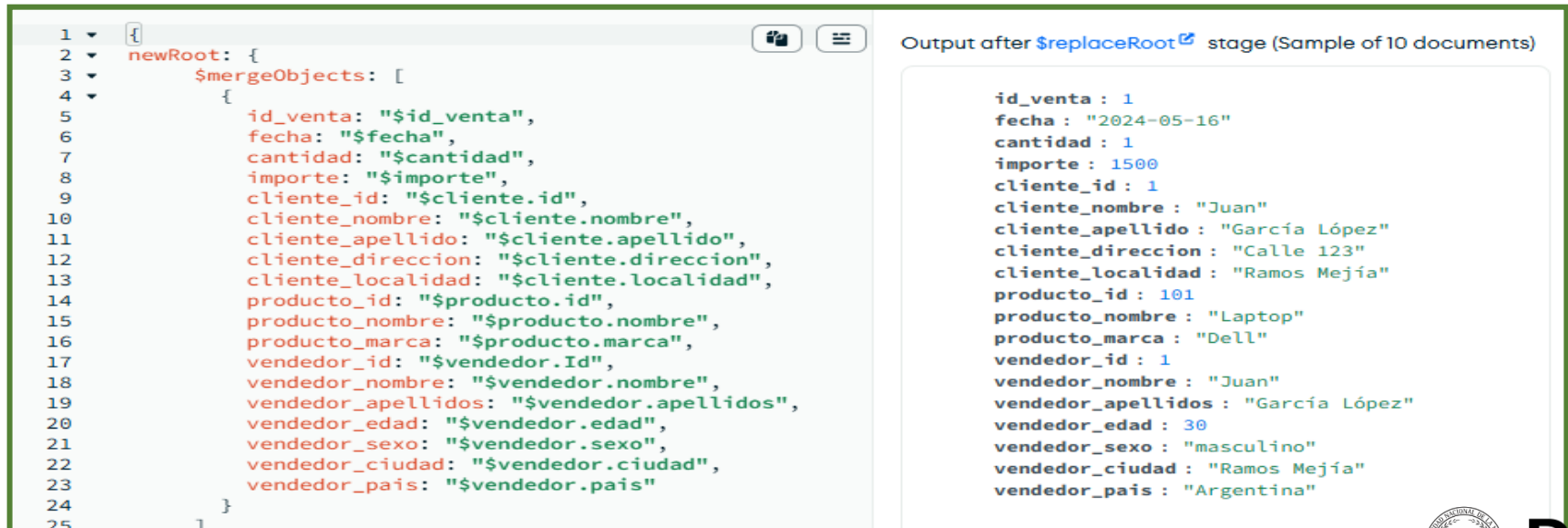
- Toma un array, y genera un documento por cada elemento del array.
- El problema es que algunos documentos:
 - ✓ pueden no tener ese campo,
 - ✓ o el array puede estar vacío.

Nota: Por defecto, `$unwind` elimina esos documentos del resultado.

- ❑ El parámetro **`preserveNullAndEmptyArrays: true`**, permite conservar esos documentos en el resultado, aun cuando el campo no exista o el array esté vacío.

\$replaceRoot

- ❑ Este operador reemplaza el documento completo actual por uno nuevo definido en el campo **newRoot**.
- ❑ Es especialmente útil cuando se quiere:
 - ✓ Reestructurar documentos complejos o anidados.
 - ✓ Fusionar subdocumentos con el documento raíz.
 - ✓ Simplificar la estructura para facilitar análisis o exportaciones.
- ❑ ¿Por qué usar **\$mergeObjects** dentro de **\$replaceRoot**?
Permite combinar múltiples subdocumentos, en un solo documento plano.
Es ideal para:
 - ✓ Crear vistas desnormalizadas.
 - ✓ Preparar datos para reportes.
 - ✓ Exportar a formatos como CSV o Excel.



```
1 {
2   newRoot: {
3     $mergeObjects: [
4       {
5         id_venta: "$id_venta",
6         fecha: "$fecha",
7         cantidad: "$cantidad",
8         importe: "$importe",
9         cliente_id: "$cliente.id",
10        cliente_nombre: "$cliente.nombre",
11        cliente_apellido: "$cliente.apellido",
12        cliente_direccion: "$cliente.direccion",
13        cliente_localidad: "$cliente.localidad",
14        producto_id: "$producto.id",
15        producto_nombre: "$producto.nombre",
16        producto_marca: "$producto.marca",
17        vendedor_id: "$vendedor.Id",
18        vendedor_nombre: "$vendedor.nombre",
19        vendedor_apellidos: "$vendedor.apellidos",
20        vendedor_edad: "$vendedor.edad",
21        vendedor_sexo: "$vendedor.sexo",
22        vendedor_ciudad: "$vendedor.ciudad",
23        vendedor_pais: "$vendedor.pais"
24      }
25    ]
26  }
27 }
```

Output after \$replaceRoot stage (Sample of 10 documents)

```
id_venta : 1
fecha : "2024-05-16"
cantidad : 1
importe : 1500
cliente_id : 1
cliente_nombre : "Juan"
cliente_apellido : "García López"
cliente_direccion : "Calle 123"
cliente_localidad : "Ramos Mejía"
producto_id : 101
producto_nombre : "Laptop"
producto_marca : "Dell"
vendedor_id : 1
vendedor_nombre : "Juan"
vendedor_apellidos : "García López"
vendedor_edad : 30
vendedor_sexo : "masculino"
vendedor_ciudad : "Ramos Mejía"
vendedor_pais : "Argentina"
```

\$replaceRoot + \$mergeObjects:

❑ Reestructuración avanzada de documentos

Documento original

```
_id: ObjectId('68272fb91a30ba135ae7ad2e')
id_venta: 1
fecha: "2024-05-16"
cantidad: 1
importe: 1500
▶ cliente: Object
  id_vendedor: 1
▶ producto: Object
▶ vendedores: Array (1)
```

\$replaceRoot junto con **\$mergeObjects** se utiliza para aplanar y unificar datos de distintos subdocumentos en un único documento final.

```
newRoot: {
  $mergeObjects: [
    {
      id_venta: "$id_venta",
      fecha: "$fecha",
      cantidad: "$cantidad",
      importe: "$importe",
      cliente_id: "$cliente.id",
      cliente_nombre: "$cliente.nombre",
      cliente_apellido: "$cliente.apellido",
      cliente_direccion: "$cliente.direccion",
      cliente_localidad: "$cliente.localidad",
      producto_id: "$producto.id",
      producto_nombre: "$producto.nombre",
      producto_marca: "$producto.marca",
      vendedor_id: "$vendedor.Id",
      vendedor_nombre: "$vendedor.nombre",
      vendedor_apellidos: "$vendedor.apellidos",
      vendedor_edad: "$vendedor.edad",
      vendedor_sexo: "$vendedor.sexo",
      vendedor_ciudad: "$vendedor.ciudad",
      vendedor_pais: "$vendedor.pais"
    }
  ]
}
```

El resultado es un documento más simple, sin objetos anidados.

Output after **\$replaceRoot** stage (Sample of 10 documents)

```
id_venta : 1
fecha : "2024-05-16"
cantidad : 1
importe : 1500
cliente_id : 1
cliente_nombre : "Juan"
cliente_apellido : "García López"
cliente_direccion : "Calle 123"
cliente_localidad : "Ramos Mejía"
producto_id : 101
producto_nombre : "Laptop"
producto_marca : "Dell"
vendedor_id : 1
vendedor_nombre : "Juan"
vendedor_apellidos : "García López"
vendedor_edad : 30
vendedor_sexo : "masculino"
vendedor_ciudad : "Ramos Mejía"
vendedor_pais : "Argentina"
```

- ❑ **\$replaceRoot** reemplaza completamente el documento actual por uno nuevo.
- ❑ Todo lo que no se incluya en **newRoot** se pierde.

\$addFields

❑ Agrega nuevos campos a los documentos o modificar campos existentes.

- Si el campo no existe, lo crea.

<pre>{ Total_Importe: { \$multiply: ["\$importe", "\$cantidad"] } }</pre>	<pre>Vendedor_ciudad : "Ramos Mejía" vendedor_pais : "Argentina" Total_Importe : 1500</pre>
---	---

- Si el campo ya existe, lo sobrescribe.

<pre>{ cliente_nombre: { \$concat: ["\$cliente_nombre", " ", "\$cliente_apellido"] } }</pre>	<pre>importe : 1500 cliente_id : 1 cliente_nombre : "Juan García López" cliente_apellido : "García López" cliente_direccion : "Calle 123" cliente_localidad : "Ramos Mejía"</pre>
---	---

- Se puede usar para calcular valores, reformatear datos, o crear campos derivados.

<pre>{ "fecha_formateada": { "\$dateToString": { "format": "%d/%m/%Y", "date": { "\$dateFromString": { "dateString": "\$fecha" } } } }}</pre>	<pre>vendedor_edad : 30 vendedor_sexo : "masculino" vendedor_ciudad : "Ramos Mejía" vendedor_pais : "Argentina" Total_Importe : 1500 mes : "Mayo" fecha_formateada : "16/05/2024"</pre>
---	---

\$addFields

❑ En este ejemplo se añade un campo nuevo que contiene el nombre del mes extraído de la fecha de la venta.

- ✓ \$let: Define variables locales (meses y mesNumero).
 - meses: Array que contiene los nombres de los meses en español.
 - mesNumero: Convierte el mes de la fecha a un número entero utilizando \$dateToString y \$dateFromString.
- ✓ in: Utiliza las variables locales para obtener el nombre del mes correspondiente al número del mes.

Esta etapa agrega un campo "mes" al documento, que representa el nombre del mes en el que se realizó la venta.

```
2  * newField: The new field name.
3  * expression: The new field expression.
4  */
5  {
6    mes: {
7      $let: {
8        vars: {
9          meses: [
10             "",
11             "Enero",
12             "Febrero",
13             "Marzo",
14             "Abril",
15             "Mayo",
16             "Junio",
17             "Julio",
18             "Agosto",
19             "Septiembre",
20             "Octubre",
21             "Noviembre",
22             "Diciembre"
23           ],
24           mesNumero: {
25             $toInt: {
26               $dateToString: {
27                 format: "%m",
28                 date: {
29                   $dateFromString: {
30                     dateString: "$fecha"
31                   }
32                 }
33             }
34           }
35         }
36       }
37     }
38   }
```

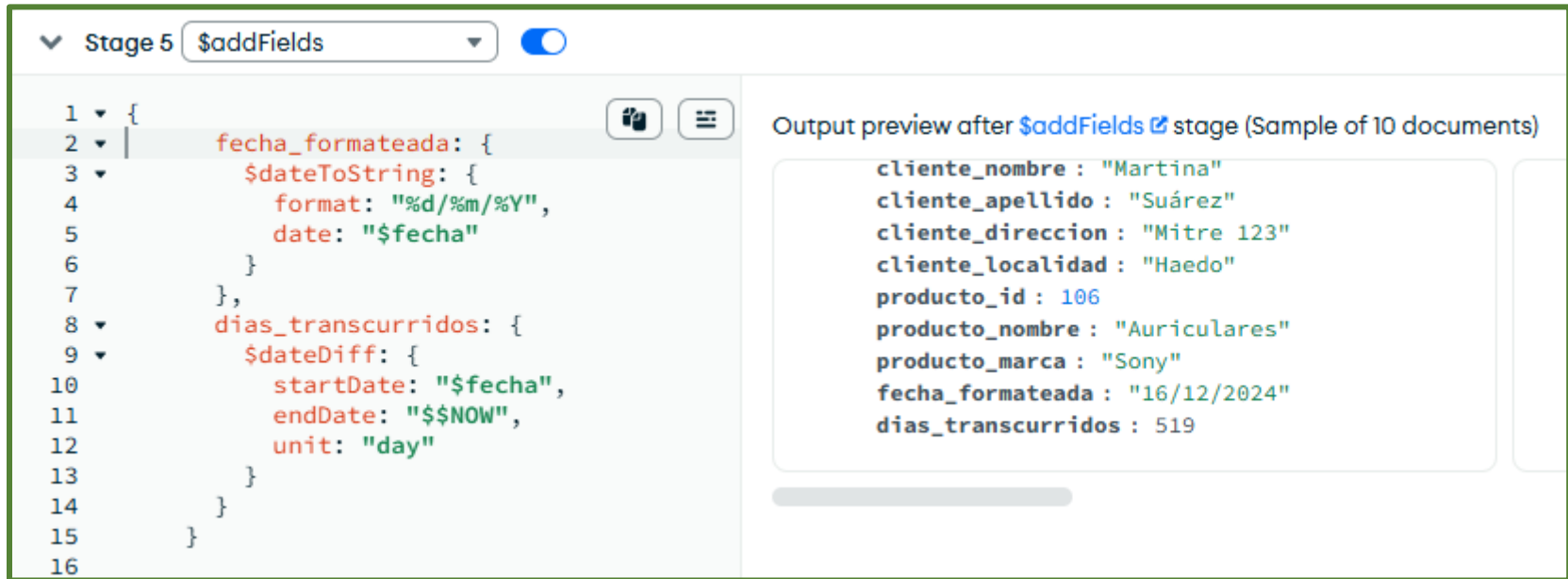
```
Output after $addFields stage (example of the document):
_id: 13
fecha: "2024-05-04"
cantidad: 1
importe: 2500
cliente_id: 1
cliente_nombre: "Juan"
cliente_apellido: "García López"
cliente_direccion: "Calle 123"
cliente_localidad: "Ramos Mejía"
vendedor_id: null
vendedor_nombre: "Juan"
vendedor_apellidos: "García López"
vendedor_edad: 30
vendedorsexo: "masculino"
vendedor_ciudad: "Ramos Mejía"
vendedor_pais: "Argentina"
producto_id: 101
producto_nombre: "Laptop"
producto_marca: "Dell"
mes: "Mayo"
```

```
_id: 5
fecha: "2024-05-12"
cantidad: 2
importe: 5000
cliente_id: 5
cliente_nombre: "Carlos"
cliente_apellido: "López"
cliente_direccion: "Avenida 789"
cliente_localidad: "Ciudad Evita"
vendedor_id: null
vendedor_nombre: "Juan"
vendedor_apellidos: "Sánchez"
vendedor_edad: 40
vendedorsexo: "masculino"
vendedor_ciudad: "Gregorio de La
vendedor_pais: "Argentina"
producto_id: 5
producto_nombre: "Disco duro ext
producto_marca: "Seagate"
mes: "Mayo"
```

Activar Windows
Ve a Configuración para activar Win

\$addFields

- ❑ Otros ejemplos: dada una fecha mostrarla de distintas maneras y determinar la cantidad de días transcurridos entre esa fecha y la actual.



The screenshot shows the MongoDB Atlas interface for a pipeline stage named `$addFields`. The stage is configured with two fields: `fecha_formateada` and `dias_transcurridos`.

```
1 {
2   |   fecha_formateada: {
3     |     $dateToString: {
4       |       format: "%d/%m/%Y",
5       |       date: "$fecha"
6     |     }
7   |   },
8   |   dias_transcurridos: {
9     |     $dateDiff: {
10      |       startDate: "$fecha",
11      |       endDate: "$$NOW",
12      |       unit: "day"
13     |     }
14   |   }
15 }
16
```

Output preview after `$addFields` stage (Sample of 10 documents)

```
cliente_nombre : "Martina"
cliente_apellido : "Suárez"
cliente_direccion : "Mitre 123"
cliente_localidad : "Haedo"
producto_id : 106
producto_nombre : "Auriculares"
producto_marca : "Sony"
fecha_formateada : "16/12/2024"
dias_transcurridos : 519
```

- ❑ Se reformatea la fecha y se calcula la cantidad de días transcurridos hasta hoy.
- ❑ `$dateDiff` devuelve un valor entero (Long).

\$addFields

- ❑ Añade otro campo calculado llamado "**Situacion**" que depende de la edad del vendedor.
 - **\$cond:**
Operador condicional que evalúa una datos: Ejemplo la edad del vendedor.
 - ✓ Si la edad es menor de 30, la situación es "**Menor de 30 años**".
 - ✓ Si la edad es exactamente 30, la situación es "**Con 30 años**".
 - ✓ Si la edad es mayor de 30, la situación es "**Mayor de 30 años**".
- ❑ Agrega una descripción de la situación del vendedor en función de su edad.

The screenshot shows the MongoDB Compass interface. On the left, the 'Stage 5' configuration for '\$addFields' is displayed. It contains a conditional expression using '\$cond' to set the 'Situacion' field based on the 'vendedor_edad' field. The logic is: if 'vendedor_edad' is less than 30, 'Situacion' is 'Menor de 30 años'; if it is 30, 'Situacion' is 'Con 30 años'; otherwise, 'Situacion' is 'Mayor de 30 años'.

On the right, the 'Output after \$addFields stage (Sample of 10 documents)' is shown. It displays two sample documents. The first document has 'vendedor_edad' 40 and 'Situacion' 'Mayor de 30 años'. The second document has 'vendedor_edad' 30 and 'Situacion' 'Con 30 años'.

```
1 {
2   Situacion: {
3     $cond: {
4       if: {
5         $lt: ["$vendedor_edad", 30]
6       },
7       then: "Menor de 30 años",
8       else: {
9         $cond: {
10          if: {
11            $eq: ["$vendedor_edad", 30]
12          },
13          then: "Con 30 años",
14          else: "Mayor de 30 años"
15        }
16      }
17    }
18  }
19 }
```

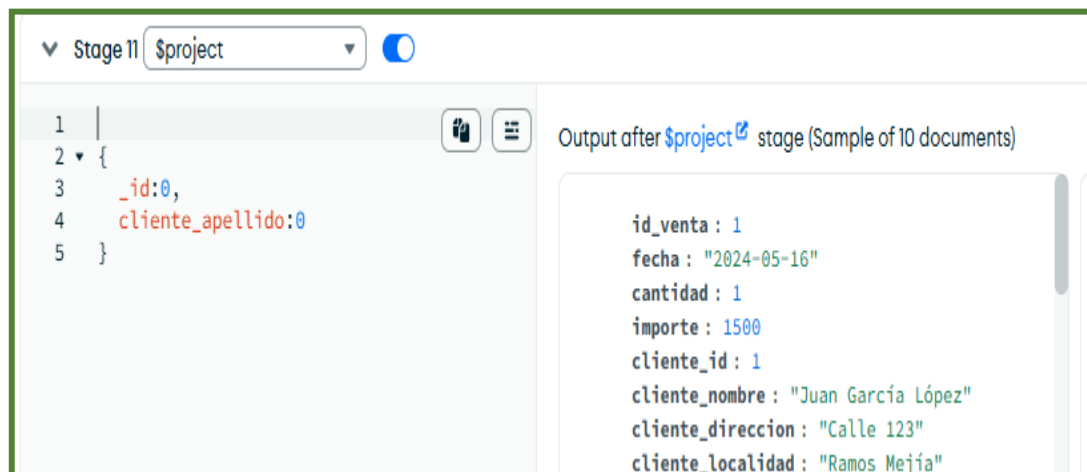
Output after \$addFields stage (Sample of 10 documents)

```
cliente_direccion: "Avenida 789"
cliente_localidad: "Ciudad Evita"
vendedor_id: null
vendedor_nombre: "Juan"
vendedor_apellidos: "Sánchez"
vendedor_edad: 40
vendedor_sexo: "masculino"
vendedor_ciudad: "Gregorio de Laferriere"
vendedor_pais: "Argentina"
producto_id: 5
producto_nombre: "Disco duro externo"
producto_marca: "Seagate"
mes: "Mayo"
Situacion: "Mayor de 30 años"

cliente_direccion: "Calle 123"
cliente_localidad: "Ramos Mejía"
vendedor_id: null
vendedor_nombre: "Juan"
vendedor_apellidos: "García López"
vendedor_edad: 30
vendedor_sexo: "masculino"
vendedor_ciudad: "Ramos Mejía"
vendedor_pais: "Argentina"
producto_id: 101
producto_nombre: "Laptop"
producto_marca: "Dell"
mes: "Mayo"
Situacion: "Con 30 años"
```


\$project

- ❑ Se utiliza para incluir o excluir campos en el resultado final.
 - **_id: 0:** Excluye el campo del resultado.
 - **cliente.apellido: 0:** Excluye este campo del resultado.

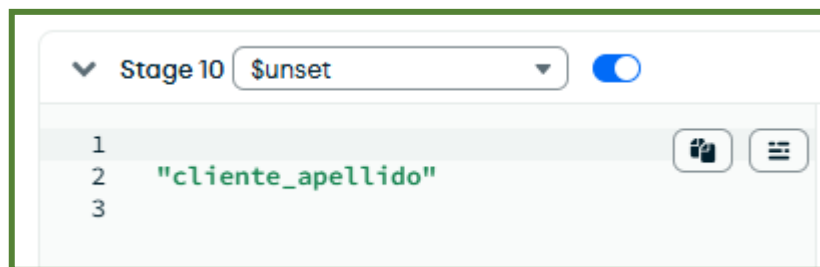


```
1 |
2 {
3   _id: 0,
4   cliente.apellido: 0
5 }
```

Output after **\$project** stage (Sample of 10 documents)

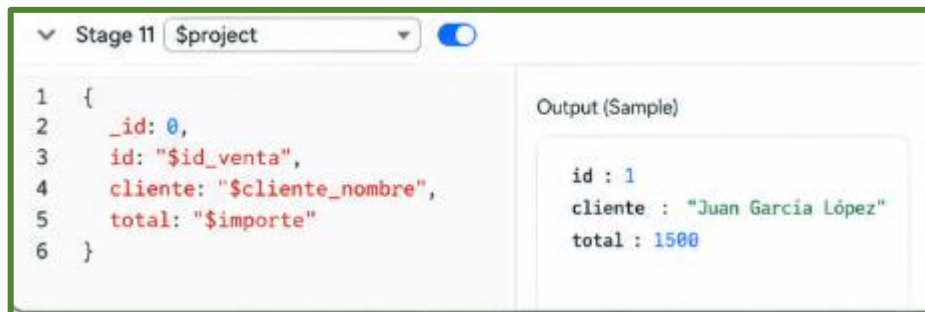
```
id_venta : 1
fecha : "2024-05-16"
cantidad : 1
importe : 1500
cliente_id : 1
cliente_nombre : "Juan García López"
cliente_direccion : "Calle 123"
cliente_localidad : "Ramos Mejía"
```

- ❑ Otra forma es usar en una agregación el comando **\$unset**. Es útil cuando solo se necesita eliminar uno o varios campos específicos sin redefinir el resto del documento.



```
1 |
2 {
3   cliente.apellido: 1
4 }
```

- ❑ **\$project** también permite **RENOMBRAR** campos.



```
1 {
2   _id: 0,
3   id: "$id_venta",
4   cliente: "$cliente_nombre",
5   total: "$importe"
6 }
```

Output (Sample)

```
id : 1
cliente : "Juan García López"
total : 1500
```


\$match

- ❑ La etapa **\$match** se utiliza para filtrar documentos dentro del pipeline de agregación según una condición específica.
- ❑ Solo los documentos que cumplen la condición continúan hacia la siguiente etapa.
- ❑ Ejemplo:

vendedor_ciudad: "Ramos Mejía":

- ✓ Filtra únicamente los documentos donde el vendedor reside en **"Ramos Mejía"**. Esta etapa ayuda a reducir la cantidad de documentos procesados, mejorando la eficiencia de la agregación.

Stage7 \$match

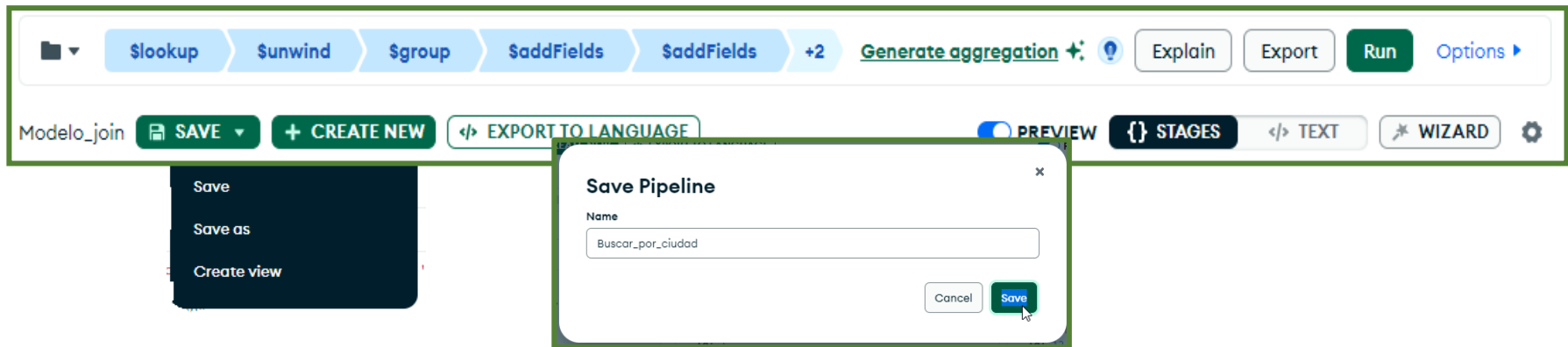
```
1 /**
2  * query: The query in MQL.
3  */
4 {
5   vendedor_ciudad: "Ramos Mejía"
6 }
```

Output after \$match stage (Sample of 7 documents)

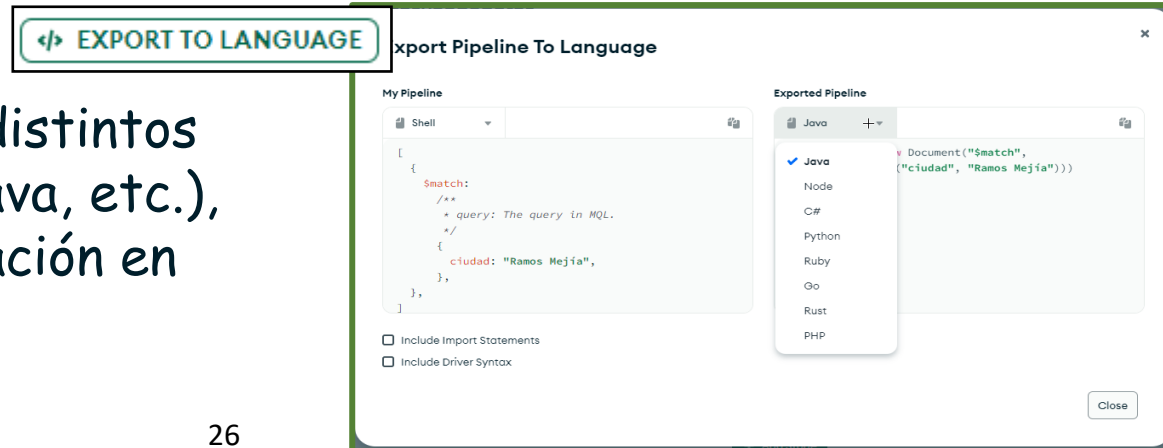
vendedor_sexo : "masculino"	vendedor_sexo : "masculino"
vendedor_ciudad : "Ramos Mejía"	vendedor_ciudad : "Ramos Mejía"
vendedor_pais : "Argentina"	vendedor_pais : "Argentina"
producto_id : 101	producto_id : 101
producto_nombre : "Laptop"	producto_nombre : "Laptop"
producto_marca : "Dell"	producto_marca : "Dell"
mes : "Mayo"	mes : "Abril"
Situacion : "Con 30 años"	Situacion : "Con 30 años"
id_venta : 13	id_venta : 37

Gestión y Exportación de Pipelines de Agregación

- ❑ Una vez creada una canalización de agregación, MongoDB permite trabajar con ella de distintas formas.
- ❑ **Guardar el pipeline:**
 - Permite almacenar la agregación para reutilizarla posteriormente, evitando tener que reconstruirla manualmente.



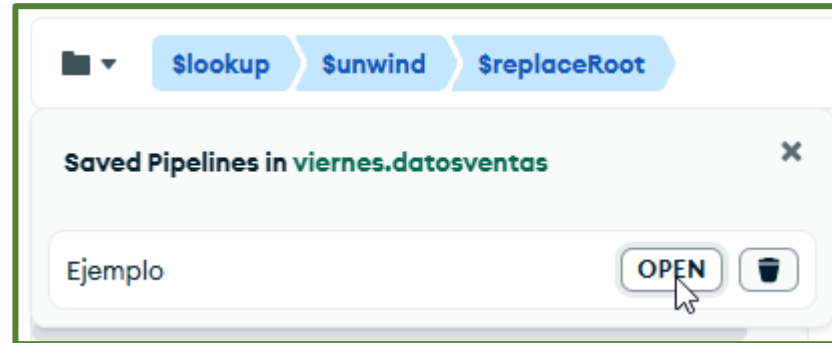
- ❑ **Exportar el pipeline:**
 - Se puede exportar a distintos lenguajes (Python, Java, etc.), facilitando su integración en aplicaciones.



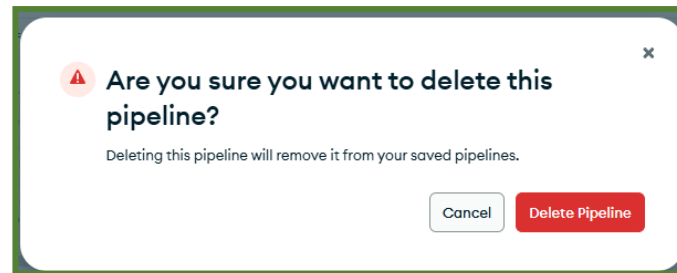
Cargar una agregación existente

❑ MongoDB permite guardar pipelines para reutilizarlos posteriormente sin necesidad de recrearlos.

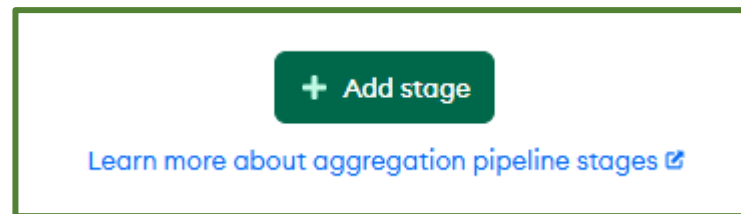
✓ En el icono de la carpeta se muestra las agregaciones guardadas.



✓ Se puede cargar (Open) o Borrarla. Donde le pedirá que confirme.

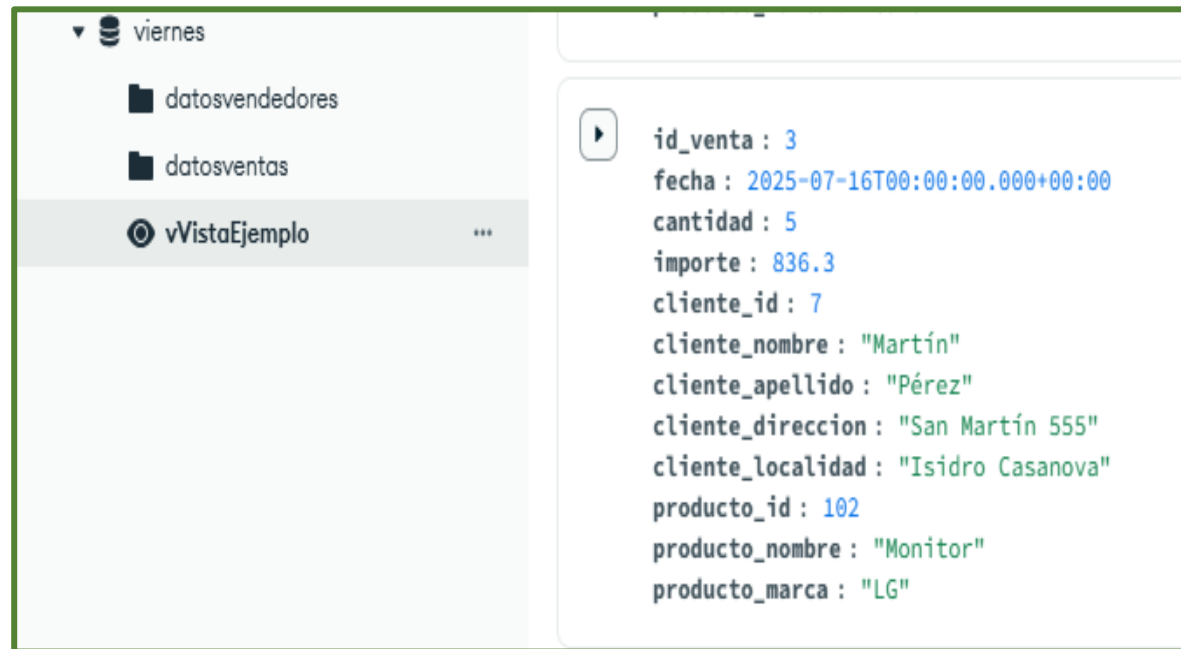
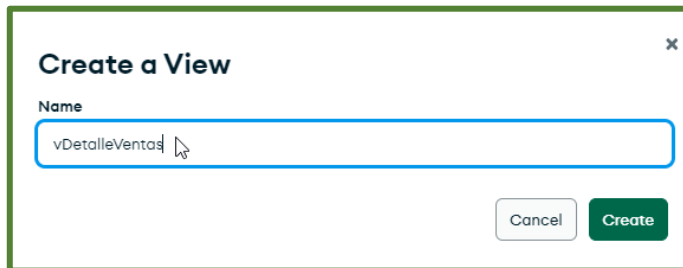
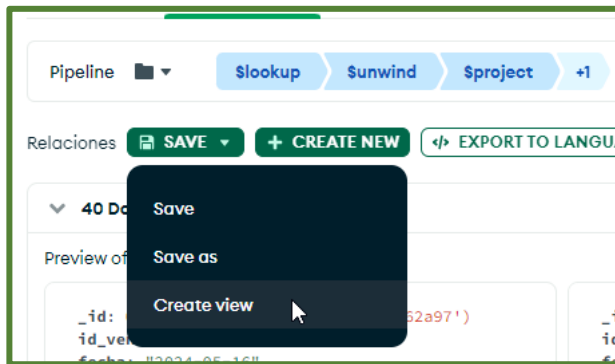


✓ Una vez cargada, se puede modificar para agregar mas etapas.



Creación de Vistas a partir de Agregaciones

- ❑ MongoDB permite guardar una agregación como una vista.
 - ✓ Es una colección virtual basada en un pipeline de agregación.
- ❑ Características:
 - ✓ - No almacena datos físicamente
 - ✓ - Se actualiza automáticamente al consultar
 - ✓ - Simplifica consultas complejas



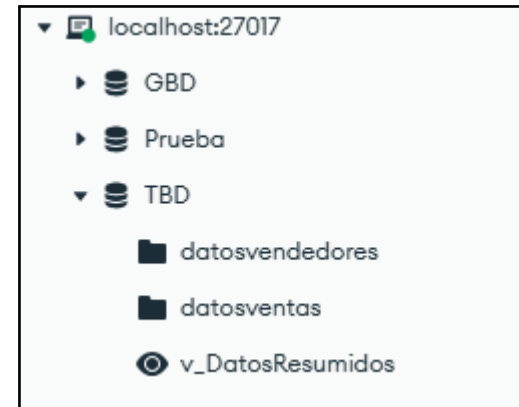
- ❑ **Nota:** Las vistas permiten trabajar con resultados de agregaciones como si fueran colecciones normales.

Crear Vistas desde el Shell de MongoDB Compass

❑ Mediante el siguiente comando se crear una vista.

`db.createView("nombreVista", "Colección" [ codigo de la agregacion ])`

```
Welcome | ventas | datosvendedores | datosventas | >_
>_MONGOSH
> db.createView(
  "vDatosResumidos",
  "datosventas",
  [
    {
      $lookup: {
        from: "datosvendedores",
        localField: "id_vendedor",
        foreignField: "Id",
        as: "vendedor"
      }
    },
    {
      $unwind: {
        path: "$vendedor",
        preserveNullAndEmptyArrays: true
      }
    },
    {
      $replaceRoot:
        /**
         * replacementDocument: A document or string.
         */
        {
          newRoot: {
            $mergeObjects: [
              {
                id_venta: "$id_venta",
                fecha: "$fecha",
                cantidad: "$cantidad",
```



Otra forma de crear una vista

- ❑ Se puede crear una vista desde el mongosh.

```
Grupo00> db.createView(  
...   "ventasxvendedores",  
...   "datosventas",  
...   [ { $lookup: {  
...     from: "datosvendedores",  
...     localField: "id_vendedor",  
...     foreignField: "Id",  
...     as: "VentasxVendedor"  
...   } },  
...   { $unwind: {  
...     path: "$VentasxVendedor",  
...     includeArrayIndex: "Subindice",  
...     preserveNullAndEmptyArrays: true  
...   } },  
...   { $replaceRoot: {  
...     newRoot: {  
...       $mergeObjects: [  
...         "$$ROOT",  
...         "$cliente",  
...         "$producto",  
...         "$VentasxVendedor" ] } } },  
...   { $project: {  
...     cliente: 0,  
...     producto: 0,  
...     VentasxVendedor: 0  
...   } },  
...   { $sort: { id_venta: 1 } }  
... ] )  
{ ok: 1 }
```

```
_id: ObjectId('66ffd1d176ae41c94bf4b0b1')  
id_venta : 1  
fecha : "2024-05-16"  
cantidad : 1  
importe : 1500  
id_vendedor : 1  
Subindice : 0  
id : 101  
nombre : "Juan"  
apellido : "García López"  
direccion : "Calle 123"  
localidad : "Ramos Mejía"  
marca : "Dell"  
Id : 1  
apellidos : "García López"  
edad : 30  
sexo : "masculino"  
ciudad : "Ramos Mejía"  
pais : "Argentina"
```

\$group

- ❑ La etapa **\$group** se utiliza para agrupar documentos según una expresión o campo común.
- ❑ Cada grupo generado produce un documento de salida identificado mediante el campo obligatorio **_id**.
- ❑ Además, permite calcular valores utilizando operadores acumuladores.

❑ Sintaxis

```
{  
  $group: {  
    _id: <expresión>,  
    <campo1>:{<variable1>:<expresión1> },  
    <campo2>:{<variable2>:<expresión2> }  
  }  
}
```

```
{  
  _id: "$producto_nombre",  
  Total_Cantidad: { $sum: 1 },  
  Total_importe: { $sum: "$importe" }  
}
```

```
_id: "Laptop"  
Total_Cantidad : 6  
Total_importe : 37075.72
```

Nota:

- * El campo **_id** es obligatorio: Si se usa **_id: null**, agrupa todos los documentos en un único documento.
- * **\$count** NO va dentro de \$group. El operador \$count es una etapa independiente del pipeline de agregación.

\$group

- Ejemplo: Total de ventas y la suma de los importes por localidad de clientes.

```
1
2 {
3   _id:null ,
4   totalIngresos: { "$sum": "$importe" },
5   ventasTotales: { "$sum": 1 }
6 }
```

Output after `$group` stage (Sample of 1 document)

```
_id: null
totalIngresos : 136500
ventasTotales : 40
```

- En MongoDB, dentro de una etapa `$group`, se debe usar `$sum: 1`, para contar.

▼ Stage1 `$group` ☒

```
1 /**
2  * _id: The id of the group.
3  * fieldN: The first field name.
4  */
5 {
6   _id: "$cliente.localidad",
7   totalventas: { $sum: "$importe" },
8   cuantasventas: { $sum: 1 }
9 }
10 }
```

Output after `$group` stage (Sam|

```
_id: "Ramos Mejía"
totalventas : 21000
cuantasventas : 7
```

```
_id: "Isidro Casanova"
totalventas : 35100
cuantasventas : 7
```

```
_id: "San Justo"
totalventas : 26400
cuantasventas : 10
```

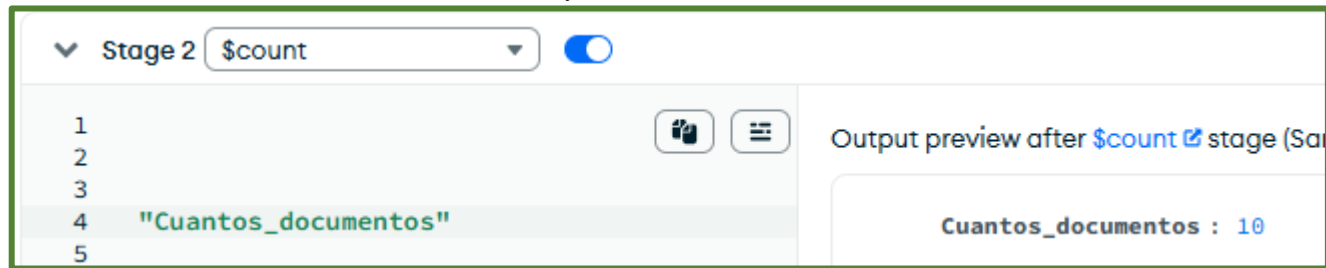
```
Grupo00> db.datosventas.aggregate([
...   {
...     $group: {
...       _id: "$cliente.localidad",
...       totalVentas: { $sum: "$importe" },
...       cantidadVentas: { $sum: 1 }
...     }
...   })
```


\$count

- ❑ La etapa **\$count** se utiliza para contar la cantidad de documentos que llegan a ese punto del pipeline.
- ❑ Devuelve un único documento con el total obtenido.
- ❑ Generalmente se utiliza después de etapas como **\$match** o **\$group**.

❑ Sintaxis

`{ $count: "nombre_del_campo" }`



```
viernes> db.vVistaEjemplo.aggregate([
...   {
...     $match: {
...       cliente_localidad: "Haedo"
...     }
...   },
...   {
...     $count: "ventas_haedo"
...   }
... ])
[ { ventas_haedo: 15 } ]
```

\$group

❑ Ejemplo con \$first y \$last

	Output after <code>\$group</code> stage (Sample of 1 document)
<pre>{ _id: null, primerProducto: { \$first: "\$producto.nombre" }, ultimoProducto: { \$last: "\$producto.nombre" }, totalIngresos: { \$sum: "\$importe" }, ventasTotales: { \$sum: 1 } }</pre>	<pre>_id: null primerProducto : "Laptop" ultimoProducto : "Mouse" totalIngresos : 136500 ventasTotales : 40</pre>

- ❑ Este ejemplo agrupa todos los documentos en un único resultado utilizando `_id: null`.
- ❑ `$first` obtiene el primer producto procesado y
- ❑ `$last` el último, según el orden de los documentos en el pipeline.
- ❑ Además, `$sum` permite calcular el total de ingresos y la cantidad total de ventas realizadas.

❑ Importante

- ✓ `$first` y `$last` dependen del orden de los documentos.
- ✓ Por eso normalmente antes se utiliza, por ejemplo:
`{ $sort: { fecha: 1 } }`

\$group

- ❑ El operador **\$push** agrega todos los productos comprados dentro de un array, conservando incluso los valores repetidos.

```
{
  _id: "$cliente.localidad",
  totalImporte: { $sum: "$importe" },
  cantidadVentas: { $sum: 1 },
  productosComprados: { $push: "$producto.nombre" }
}
```

Output after \$group stage (Sample of 6 documents)

```
_id: "Ramos Mejía"
totalImporte: 21000
cantidadVentas: 7
productosComprados: Array (7)
0: "Laptop"
1: "Router"
2: "Laptop"
3: "Router"
4: "Laptop"
5: "Router"
```

- ❑ Este ejemplo agrupa las ventas por localidad del cliente utilizando `_id: "$cliente.localidad"`.
- ❑ **\$addToSet** agrega valores dentro de un array, pero evitando elementos duplicados. Es decir:
 - ✓ si un valor aparece varias veces,
 - ✓ solo se guarda una única vez en el resultado.

```
{
  _id: "$cliente.localidad",
  totalImporte: { $sum: "$importe" },
  cantidadVentas: { $sum: 1 },
  productosComprados: { $addToSet: "$producto.nombre" }
}
```

```
_id: "Ramos Mejía"
totalImporte: 21000
cantidadVentas: 7
productosComprados: Array (2)
0: "Router"
1: "Laptop"
```

Etapas de filtrado y agrupamiento

❑ \$group

- Ejemplo: Ventas por producto y marcas ordenado por producto y marca

```
Stage1 $group
1 {
2   _id: null,
3   idProducto: { $first: "$producto.id"
4   marca: { $first: "$producto.marca"
5   totalIngresos: { $sum: "$importe" }
6   ventasTotales: { $sum: 1 }
7 }
```

```
Stage2 $sort
1 {
2 {
3   "_id.idProducto": 1, "_id.marca": 1
4 }
```

```
Grupo00> db.datosventas.aggregate([
... {
...   $group: {
...     _id: { idProducto: "$producto.id", marca: "$producto.marca" },
...     totalIngresos: { $sum: "$importe" },
...     ventasTotales: { $sum: 1 }
...   },
...   $sort: { "_id.idProducto": 1, "_id.marca": 1 }
... }
... ])
[
  {
    _id: { idProducto: 4, marca: 'HP' },
    totalIngresos: 1200,
    ventasTotales: 1
  },
  {
    _id: { idProducto: 5, marca: 'Seagate' },
    totalIngresos: 5000,
    ventasTotales: 1
  },
]
```

Output after \$sort stage (Sample of 10 documents)

```
▼ _id: Object
  idProducto: 4
  marca: "HP"
  totalIngresos : 1200
  ventasTotales : 1
```

```
▼ _id: Object
  idProducto: 5
  marca: "Seagate"
  totalIngresos : 5000
  ventasTotales : 1
```

Etapas de filtrado y agrupamiento

❑ \$group

Se agrupan todos los documentos por el campo "id_venta" y se selecciona los primeros valores encontrados para cada campo especificado, creando un solo documento por cada venta con todos los detalles de la venta y del vendedor..

```
1 {
2
3   _id: "$producto.id",
4   productoNombre: {
5     $first: "$producto.nombre" },
6   minImporte: { $min: "$importe" },
7   maxImporte: { $max: "$importe" }
8 }
9 }
```

Output after \$group stage (Sample of 10 documents)

```
_id: 111
productoNombre: "Teclado inalámbrico"
minImporte: 2000
maxImporte: 2000
```

```
Grupo00> db.datosventas.aggregate([
...   {
...     $group: {
...       _id: "$producto.id",
...       productoNombre: { $first: "$producto.nombre" },
...       minImporte: { $min: "$importe" },
...       maxImporte: { $max: "$importe" }
...     }
...   }
... ])
[
  {
    _id: 103,
    productoNombre: 'Teclado',
    minImporte: 2700,
    maxImporte: 4500
  },
  ...
]
```

\$group y Shell

- ❑ En el Mongo Shell, se utiliza dentro de **`db.coleccion.aggregate([ ... ])`**.
- ❑ Cada grupo se identifica con el campo `_id`, que puede ser un solo valor o una expresión compuesta.
- ❑ A través de acumuladores ya mencionados, se pueden obtener totales, estadísticas o colecciones de valores.
- ❑ Ejemplos prácticos en shell:

```
Grupo00> db.datosventas.aggregate([
...   {
...     $group: {
...       _id: "$producto.id",
...       productoNombre: { $first: "$producto.nombre" },
...       minImporte: { $min: "$importe" },
...       maxImporte: { $max: "$importe" }
...     }
...   }
... ])
[
  {
    _id: 103,
    productoNombre: 'Teclado',
    minImporte: 2700,
    maxImporte: 4500
  },

```

```
Grupo00> db.datosventas.aggregate([
...   {
...     $group: {
...       _id: "$producto.id",
...       productoNombre: { $first: "$producto.nombre" },
...       importes: { $push: "$importe" }
...     }
...   }
... ])
[
  { _id: 104, productoNombre: 'Mouse', importes: [ 2200, 2200, 2200 ] },
  {
    _id: 102,
    productoNombre: 'Monitor',
    importes: [ 3000, 4000, 4000, 4000 ]
  },

```